

Módulo extra

Servidores MCP para tus proyectos: construcción e integración con Claude Code y ChatGPT

Desarrollo de Software IX · Código 1493 · Actualizado a junio 2026

Propósito del módulo

Explicar el Model Context Protocol y guiar la construcción de un servidor MCP sobre un proyecto propio — el comercio electrónico del Parcial #2 — y su integración en Claude Code y ChatGPT. Este material es la base del próximo proyecto.

1. Panorama: por qué existe MCP

En el módulo extra de agentes vimos cómo Claude Code, Codex y otros agentes ayudan a construir software. Este módulo invierte la pregunta: ¿cómo logramos que esos agentes puedan operar NUESTRO software? La respuesta de la industria es el Model Context Protocol (MCP), un protocolo abierto publicado por Anthropic en noviembre de 2024 y adoptado desde 2025 por OpenAI, Google y el resto del ecosistema. MCP estandariza cómo una aplicación de IA descubre y usa capacidades externas: datos, herramientas y acciones.

Antes de MCP, conectar un asistente con un sistema externo exigía una integración a la medida por cada par asistente-sistema: N asistentes por M sistemas = $N \times M$ integraciones. MCP reduce el problema a $N+M$: cada asistente implementa el protocolo una vez como cliente, cada sistema lo implementa una vez como servidor, y cualquier combinación funciona. Es la misma jugada que estandarizó los puertos de hardware — por eso se le apoda el "USB-C de las aplicaciones de IA".

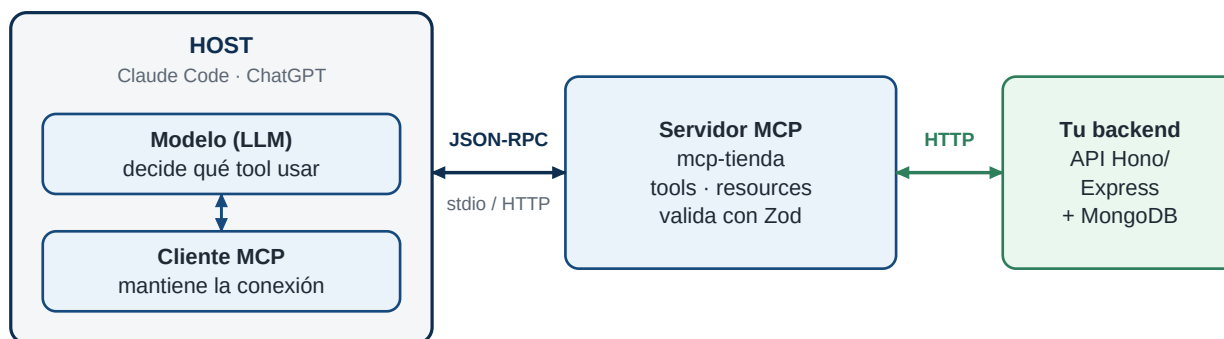
El proyecto que sigue

Este material es la base del próximo proyecto del curso: construir un servidor MCP sobre el comercio electrónico que desarrollaste en el Parcial #2, e integrarlo en Claude Code y ChatGPT, de modo que un agente pueda consultar tu catálogo, revisar órdenes y operar tu tienda conversando. El enunciado formal se publicará en Teams.

2. Arquitectura del protocolo

2.1 Host, cliente y servidor

MCP define tres roles. El HOST es la aplicación de IA con la que interactúa el usuario (Claude Code, ChatGPT, un IDE). Dentro del host vive un CLIENTE MCP que mantiene la conexión. Del otro lado está el SERVIDOR MCP: un programa — frecuentemente pequeño — que expone capacidades de un sistema concreto. El modelo de lenguaje nunca toca tu base de datos: pide al host ejecutar una herramienta, el cliente la invoca en el servidor, y el servidor decide qué hacer y qué responder. La comunicación usa JSON-RPC 2.0.

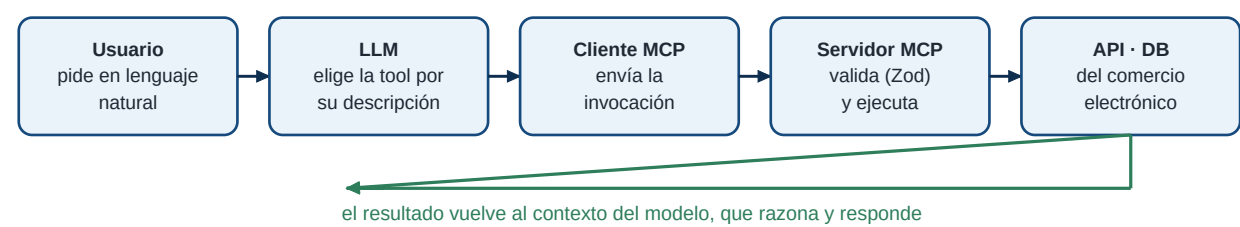


El modelo nunca toca la base de datos: pide; el servidor decide y responde.

2.2 Las tres primitivas

Primitiva	Qué es	Ejemplo en el comercio electrónico
Tools	Acciones que el MODELO decide invocar: funciones con nombre, descripción y esquema de entrada.	buscar_productos, crear_orden, estado_orden, resumen_ventas.
Resources	Datos de solo lectura que la APLICACIÓN puede adjuntar como contexto.	El catálogo en JSON, la política de envíos, el esquema de la base.
Prompts	Plantillas reutilizables que el USUARIO invoca explícitamente.	"Analiza las ventas de la semana y sugiere promociones".

La distinción importa para el diseño: una herramienta es algo que el modelo puede decidir ejecutar por su cuenta dentro de una conversación; un recurso es contexto que se aporta; un prompt es un atajo de usuario. La mayor parte del valor de un MCP de proyecto está en las tools bien diseñadas.



2.3 Transportes: stdio y HTTP

Transporte	Cómo corre el servidor	Cuándo usarlo
stdio	El host lanza el servidor como proceso local y conversan por entrada/salida estándar.	Desarrollo local y Claude Code: simple, sin red, sin autenticación adicional.
Streamable HTTP	El servidor corre como servicio web (propio o en la nube) y los clientes se conectan por HTTPS.	Integraciones remotas: ChatGPT solo acepta servidores remotos; permite compartir un MCP entre varios usuarios.

3. Diseñar el MCP de tu proyecto

Antes del código viene una decisión de diseño: ¿qué operaciones del comercio electrónico merecen ser herramientas? La tentación es exponer la API completa; el buen diseño expone un conjunto pequeño de operaciones con INTENCIÓN clara, porque el modelo elige qué herramienta usar leyendo sus nombres y descripciones — una herramienta ambigua se usa mal o no se usa.

- Nombra por intención del dominio, no por endpoint: buscar_productos mejor que get_api_products_v1.
- Describe para el modelo: la descripción debe decir qué hace, qué espera y qué devuelve. Es documentación que un LLM va a leer literalmente.
- Esquemas estrictos: cada parámetro con tipo y validación (Zod). El modelo comete menos errores cuando el contrato es preciso.
- Separa lectura de escritura: consultar el catálogo es inocuo; crear órdenes o cambiar precios merece confirmación o quedar fuera de la primera versión.

- Respuestas compactas y útiles: el resultado vuelve al contexto del modelo; una lista de 500 productos completos desborda — devuelve los 10 más relevantes con los campos que importan.

Reutiliza tu backend

El servidor MCP no reemplaza tu API REST: la consume. Cada tool llama a los endpoints (o servicios) que ya construiste con Hono/Express y MongoDB. Así la lógica de negocio sigue viviendo en un solo lugar y el MCP es una capa delgada de traducción — el mismo principio de adaptadores del Capítulo 9.

4. Construir el servidor con el SDK de TypeScript

4.1 Proyecto y dependencias

```
bun init mcp-tienda
bun add @modelcontextprotocol/sdk zod
```

4.2 Servidor con una tool de lectura

```
// src/index.ts
import { McpServer } from '@modelcontextprotocol/sdk/server/mcp.js'
import { StdioServerTransport } from '@modelcontextprotocol/sdk/server/stdio.js'
import { z } from 'zod'

const server = new McpServer({ name: 'mcp-tienda', version: '1.0.0' })

server.registerTool(
  'buscar_productos',
  {
    title: 'Buscar productos',
    description: 'Busca productos del catálogo por texto y precio máximo. ' +
      'Devuelve hasta 10 resultados con id, nombre, precio y stock.',
    inputSchema: {
      consulta: z.string().describe('Texto a buscar en nombre y descripción'),
      precioMax: z.number().optional().describe('Precio máximo en USD'),
    },
  },
  async ({ consulta, precioMax }) => {
    const url = new URL(`${process.env.TIENDA_API}/api/products`)
    url.searchParams.set('q', consulta)
    if (precioMax) url.searchParams.set('maxPrice', String(precioMax))
    const res = await fetch(url)
    const productos = await res.json()
    return {
      content: [{ type: 'text', text: JSON.stringify(productos.slice(0, 10)) }],
    }
  }
)

await server.connect(new StdioServerTransport())
```

El patrón se repite para cada herramienta: `registerTool` recibe el nombre, los metadatos (título, descripción y esquema Zod de entrada) y el manejador `async` que ejecuta la operación — aquí, llamando a la API del proyecto — y devuelve contenido. Una tool de escritura como `crear_orden` sigue la misma forma, validando con Zod los items y devolviendo el identificador y total de la orden creada.

4.3 Probar con el Inspector

Antes de conectar ningún asistente, el MCP Inspector — la herramienta oficial de depuración — permite lanzar el servidor, listar sus tools, invocarlas a mano y ver las respuestas crudas:

```
npx @modelcontextprotocol/inspector bun src/index.ts
```

El flujo de prueba mínimo: verificar que las tools aparecen con sus descripciones, invocar cada una con entradas válidas e inválidas (el esquema debe rechazar las inválidas), y confirmar que las respuestas son compactas y correctas contra la base real del proyecto.

5. Integración en Claude Code

Claude Code consume servidores MCP locales por stdio. Hay dos formas de registrarlo. La rápida, desde la terminal:

```
claude mcp add tienda -e TIENDA_API=http://localhost:3000 -- bun src/index.ts
```

La recomendada para el proyecto: un archivo `.mcp.json` en la raíz del repositorio, que se versiona en Git y hace que cualquier integrante del equipo (y el profesor al clonar) tenga el servidor disponible automáticamente:

```
// .mcp.json (raíz del repo)
{
  "mcpServers": {
    "tienda": {
      "command": "bun",
      "args": ["mcp/src/index.ts"],
      "env": { "TIENDA_API": "http://localhost:3000" }
    }
  }
}
```

Dentro de Claude Code, el comando `/mcp` muestra los servidores conectados y sus herramientas. A partir de ahí, la conversación opera la tienda: "¿qué productos tienen stock bajo 5?", "crea una orden de prueba con dos unidades del producto X", "resume las ventas de esta semana" — el agente elige las tools, las invoca y razona sobre los resultados.

6. Integración en ChatGPT

ChatGPT integra MCP a través de sus apps/connectors (los connectors fueron renombrados a apps en diciembre de 2025). A diferencia de Claude Code, ChatGPT NO lanza procesos locales: exige un servidor MCP REMOTO accesible por HTTPS con transporte Streamable HTTP. El camino:

- 1 Adaptar el servidor al transporte HTTP (el SDK trae `StreamableHTTPServerTransport`; el servidor pasa a ser una app web que puede convivir con tu API Hono).
- 2 Desplegarlo con HTTPS público (el mismo proveedor donde vive tu API, o un túnel de desarrollo para demos).
- 3 En ChatGPT: Settings → Apps & Connectors → activar Developer mode (disponible en planes Pro/Plus y Enterprise/Edu según la organización) → Create app/connector con la URL del servidor.

- 4 Probar en una conversación habilitando la app: ChatGPT lista las tools del servidor y las invoca igual que Claude.

Si tu plan de ChatGPT no permite connectors de desarrollador

La integración con ChatGPT puede demostrarse con una cuenta del equipo que tenga Developer mode, o sustituirse en la demo por el MCP Inspector apuntando al servidor remoto — lo que evalúa el proyecto es que el servidor hable MCP por HTTP correctamente, no la suscripción.

Un mismo código de servidor puede soportar ambos transportes: stdio para el trabajo local con Claude Code y HTTP para la integración remota — el núcleo (las tools) no cambia, solo la capa de conexión.

7. Seguridad del servidor MCP

- El servidor corre con TUS credenciales: el modelo no ve la llave de la base ni los secretos, pero todo lo que una tool permita, el modelo lo puede hacer. El control de daños se diseña en las tools, no en el prompt.
- Tools de escritura con freno: montos máximos, estados permitidos, modo prueba de la pasarela. Una tool crear_orden jamás debe poder cobrar una tarjeta real en la demo.
- Un servidor remoto sin autenticación es una API pública: protege el endpoint HTTP (token Bearer u OAuth) igual que protejiste las rutas privadas con JWT en el Capítulo 5.
- Valida las entradas con los esquemas Zod y trata las salidas hacia el modelo como datos, no como instrucciones: contenido malicioso en una descripción de producto podría intentar manipular al agente (inyección de prompt).
- Principio de mínima exposición: empieza con tools de lectura; agrega escritura solo donde el proyecto lo justifique y documenta cada una en el README.

8. Lo que se espera del próximo proyecto

Como base de preparación — el enunciado formal con fechas, puntos y rúbrica se publicará en Teams — el proyecto consistirá en llevar tu comercio electrónico del Parcial #2 a la era agéntica:

Componente	Expectativa
Servidor MCP	Construido con @modelcontextprotocol/sdk + Zod sobre TU comercio electrónico, consumiendo tu API existente (no lógica duplicada).
Tools	Un conjunto pequeño y bien descrito que cubra lectura (catálogo, órdenes, métricas) y al menos una operación de escritura controlada.
Pruebas	Evidencia con MCP Inspector: tools listadas, invocaciones válidas e inválidas.
Integración Claude Code	.mcp.json versionado en el repo y demo conversacional operando la tienda.
Integración ChatGPT	Servidor con transporte HTTP desplegado y conectado como app/connector (o demo equivalente vía Inspector remoto).
Documentación	README: arquitectura, tools con sus contratos, cómo correr local y remoto, y decisiones de seguridad.

9. Actividades sugeridas

- Levanta tu comercio electrónico del Parcial #2 y lista en papel las 5 operaciones más valiosas para un agente; clasifícalas en lectura/escritura.
- Implementa `buscar_productos` y pruébala en el Inspector con entradas válidas e inválidas.
- Conecta el servidor a Claude Code con `.mcp.json` y pide en conversación un análisis del catálogo que requiera 2 tools encadenadas.
- Adapta el servidor a Streamable HTTP y conéctalo (a ChatGPT con Developer mode, o al Inspector por URL).
- Escribe la sección de seguridad del README: qué expusiste, qué no, y por qué.

10. Cierre y referencias

MCP cierra el arco del curso: construiste una aplicación Full Stack (frontend, backend, datos, pagos), aprendiste a desarrollarla con agentes, y ahora la conviertes en algo que los agentes pueden operar. Esa es la forma 2026 de "integrar un sistema": no solo exponer una API para otros programas, sino un contrato que las IAs entienden.

Referencias: modelcontextprotocol.io (especificación y SDKs) ·

github.com/modelcontextprotocol/typescript-sdk · docs de Claude Code (sección MCP) ·

developers.openai.com (Apps SDK y connectors MCP) · Capítulo 9 (integraciones externas) y módulo extra de agentes de este curso. Corte de actualidad: junio 2026 — el ecosistema MCP evoluciona rápido; verificar las docs oficiales al implementar.